

# Mastering Actor-Critic & GAE

Complete Beginner-Friendly Breakdown: Value Bootstrapping, Temporal Difference Errors, GAE  
Telescoping Derivation, Squashed Gaussians & Compliant Slicing

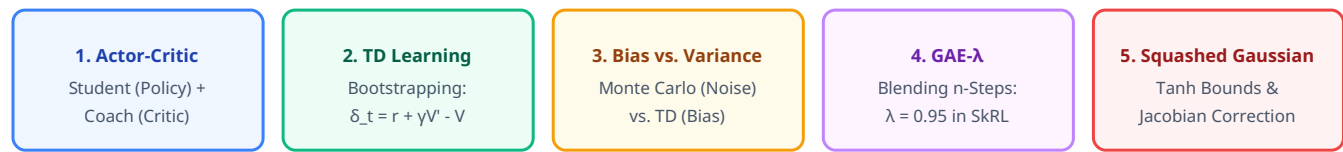
**Instructor:** Prof. Sergey Levine (UC Berkeley)    **Companion:** DEX-ROB Lab, Tianjin University    **Frameworks:** Achiam (Spinning Up) + Schulman & Levine GAE

## 0. The "Mental Map": Why Does Lecture 6 Exist?

In Lecture 5, we discovered the Policy Gradient Theorem and learned that we can reduce variance by subtracting a baseline  $b(s) = V(s)$ . However, in REINFORCE, we still had to wait until the robot completed an entire 200-step cutting trajectory to calculate empirical reward-to-go  $\hat{Q}_{i,t} = \sum_{t'=t}^T r_{t'}$ .

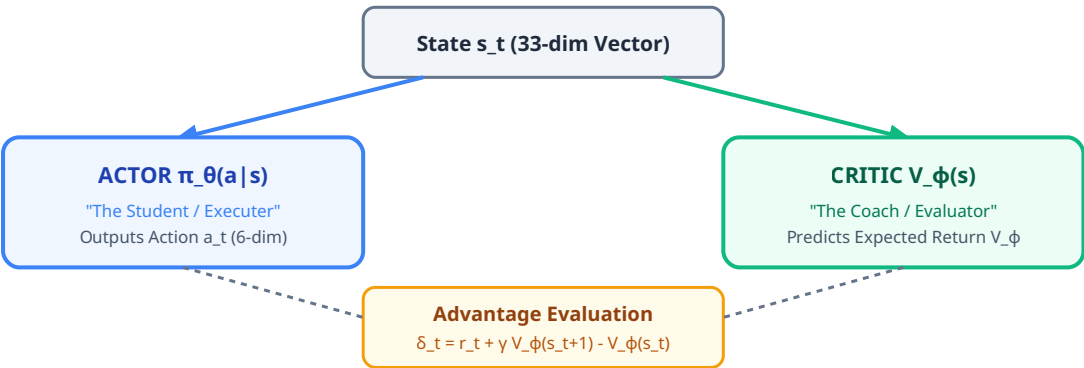
In continuous physical robotics, relying on full trajectory rollouts is dangerous: a single unexpected contact slip or numerical perturbation at step 195 corrupts the return for all previous 194 steps.

**Lecture 6 solves this by introducing Actor-Critic architectures.** Instead of waiting for the future to happen, the robot trains a second neural network—the **Critic**—to predict the future. The Actor can then update its actions at every individual time-step using *bootstrapping* and *Temporal Difference (TD) learning*.



## Part 1: The Core Architecture: Actor (Student) & Critic (Coach)

(Slides 1–15, Spoken Transcript 03:10–18:25) An Actor-Critic algorithm divides the learning agent into two distinct neural networks that work in symbiosis:



## 1.1 Dissecting the 3 Core Value Notations

| Function                  | Notation & Formula                                                                               | Physical Meaning in Tomato Slicing                                                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>State Value</b>        | $V^\pi(s) = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$             | How promising is the current situation? E.g., <i>"The knife is resting flush on the cuticle with zero tilt error. Expected return is +35."</i>                  |
| <b>State-Action Value</b> | $Q^\pi(s, a) = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$ | How good is taking a specific action in this state? E.g., <i>"In this state, if I choose action <math>a_1</math> (saw at 25 mm/s), expected return is +42."</i> |
| <b>Advantage Function</b> | $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$                                                           | How much better was this action compared to the average policy action?<br>$A = 42 - 35 = +7$ (Better than expected!).                                           |

## Part 2: Value Function Fitting & Temporal Difference Bootstrapping

(Slides 16–35, Spoken Transcript 18:30–36:00) How does the Critic learn to predict  $V(s)$ ?

### 2.1 Monte Carlo vs. Temporal Difference (TD)

1. **Monte Carlo Target:** Run full episode until  $t = T$ , sum all observed rewards  $y_t^{\text{MC}} = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ .

$$\min_{\phi} \frac{1}{2} \sum_t (V_{\phi}(s_t) - y_t^{\text{MC}})^2$$

*Flaw:* Unbiased, but monstrous variance because  $y_t^{\text{MC}}$  depends on hundreds of stochastic future steps.

2. **Temporal Difference (TD) Bootstrapping Target:** Observe just *one* step: get immediate reward  $r_t$  and bootstrap from  $V(s_{t+1})$ :

$$y_t^{\text{TD}} = r_t + \gamma V_{\phi}(s_{t+1})$$

The regression loss minimizes the **Bellman error**:

$$\min_{\phi} \frac{1}{2} \sum_t (V_{\phi}(s_t) - [r_t + \gamma V_{\phi}(s_{t+1})])^2$$

### 2.2 The Temporal Difference Error ( $\delta_t$ )

$$\delta_t^V = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

Notice that  $\delta_t^V$  is a 1-step sample estimate of the Advantage function:  $Q(s_t, a_t) \approx r_t + \gamma V(s_{t+1}) \implies A(s_t, a_t) \approx \delta_t^V$ .

#### SPINNING UP BUG ALERT: SEMI-GRADIENT DETACH TRAP

Joshua Achiam warns: Bellman updates are **semi-gradient methods**. When computing MSE loss for the Critic: `loss = (V(s) - (r + gamma * V(s_next).detach()))**2` If you omit `.detach()` on `V(s_next)`, PyTorch will compute gradients with respect to both terms. This is mathematically broken and leads to runaway gradient divergence.

## Part 3: The Bias-Variance Dilemma & N-Step Returns

(Slides 36–48, Spoken Transcript 36:15–48:40) Neither pure Monte Carlo nor 1-step TD is fully satisfactory:

- **Monte Carlo ( $\infty$ -step):** Zero bias, but extreme variance. Soft tissue contact noise compounds endlessly.
- **1-Step TD:** Minimal variance, but biased if the Critic's prediction is imperfect early in training.

- **N-Step Return:** Looks  $n$  steps ahead before bootstrapping:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$$

## Part 4: Generalized Advantage Estimation (GAE- $\lambda$ )

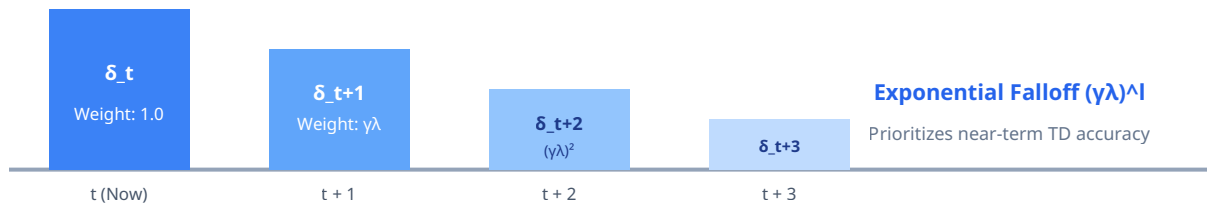
(Slides 49–65, Spoken Transcript 48:50–1:05:20) Schulman, Levine et al. (2016) proposed taking an exponentially weighted average of all  $k$ -step advantage estimators:

### 4.1 The Telescoping Sum Derivation

Define the  $k$ -step advantage:  $\hat{A}_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}) - V(s_t) = \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}^V$ . Notice how intermediate value terms telescope and cancel! GAE takes an exponentially weighted sum across all horizons using parameter  $\lambda \in [0, 1]$ :

**The GAE Advantage Formula:**

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \sum_{k=1}^{\infty} \lambda^{k-1} \hat{A}_t^{(k)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$



### 4.2 The Magic Parameter Tuning for Robotics

- $\lambda = 0$ :  $\hat{A}_t = \delta_t^V$ . Pure 1-step TD. Minimal variance, highest bias.
- $\lambda = 1$ :  $\hat{A}_t = \sum \gamma^l r_{t+l} - V(s_t)$ . Pure Monte Carlo. Zero bias, highest variance.
- **The Universal Gold Standard:**  $\lambda = 0.95, \gamma = 0.99$ . In Isaac Lab and SkRL, setting  $\lambda = 0.95$  provides the sweet spot: it allows the policy to look roughly 20–40 steps ahead while aggressively damping out long-term contact noise.

## Part 5: Continuous Control & Squashed Gaussian Policies

(Slides 66–78 & Spinning Up ch19) In continuous robotics, actions are physically bounded (e.g., motor velocities cannot exceed physical limits). How do we bound continuous Gaussian policies?

### 5.1 The Tanh Squashing Transformation

Instead of clipping actions (which destroys gradients outside the boundary), we apply a smooth hyperbolic tangent squashing function:  $a = \tanh(u)$ , where  $u \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$ .

### 5.2 The Jacobian Determinant Correction

Because  $\tanh$  is a non-linear change of variables, the probability density changes! By the change of variables formula:

$$P(a|s) = P(u|s) \cdot \left| \det \left( \frac{da}{du} \right) \right|^{-1}$$

Taking the logarithm:

$$\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^d \log (1 - \tanh^2(u_i) + \epsilon)$$

Omitting this Jacobian correction term is a classic silent failure that distorts entropy calculations and policy gradient updates!

## Part 6: Paper 1 PyTorch Implementation of GAE

```
import torch
def compute_gae(rewards, values, next_values, dones, gamma=0.99, lam=0.95):
    """ Vectorized computation of Generalized Advantage Estimation (GAE-λ).
    rewards: [T, num_envs]
    values: [T, num_envs] - Critic predictions V(s_t)
    next_values: [T, num_envs] - Critic predictions V(s_{t+1})
    dones: [T, num_envs] - Episode termination flags """
    num_steps = rewards.size(0)
    advantages = torch.zeros_like(rewards)
    last_gae = 0.0
    # Iterate backwards in time (from T-1 down to 0)
    for t in reversed(range(num_steps)):
        non_terminal = 1.0 - dones[t]
        # 1-Step TD Error: δ_t = r_t + γ V(s_{t+1}) - V(s_t)
        delta = rewards[t] + gamma * next_values[t] * non_terminal - values[t]
        # Recursive GAE accumulation: A_t = δ_t + (γ λ) A_{t+1}
        advantages[t] = last_gae = delta + gamma * lam * non_terminal * last_gae
    # Target values for training the Critic: Returns = Advantage + V(s)
    returns = advantages + values
    # Normalize advantages across the parallel environment batch
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
    return advantages, returns
```

## Part 7: Interactive Tablet Self-Test Quiz (Test Your Understanding)

**Question 1: What happens to GAE when you set  $\lambda = 0$ , and what happens when you set  $\lambda = 1$ ?**

**Answer:** When  $\lambda = 0$ , GAE reduces strictly to 1-step TD error ( $\delta_t = r_t + \gamma V(s') - V(s)$ ), giving lowest variance but highest bias. When  $\lambda = 1$ , GAE telescopes into the full empirical Monte Carlo return minus baseline ( $\sum \gamma^l r_{t+l} - V(s)$ ), giving zero bias but maximum variance.

**Question 2: Why must the Critic network be trained with semi-gradients (detaching the target)?**

**Answer:** The Bellman target  $y = r + \gamma V(s')$  is derived from dynamic programming, where  $V(s')$  serves as a fixed reference value. If we backpropagate gradients into  $V(s')$ , the network will try to move both its current prediction and future target simultaneously, causing eigenvalues to drift and resulting in mathematical instability.

**Question 3: Why does setting  $\lambda = 0.95$  prevent the robot from misinterpreting a skin puncture reward?**

**Answer:** Skin puncture is an abrupt transition that yields a high localized reward. Setting  $\lambda = 0.95$  focuses the advantage assignment on the 20–40 steps directly surrounding the rupture event, rewarding the preparatory deceleration and sawing actions without smearing noise across the subsequent 160 steps of pulp slicing.

## Part 8: Thesis Defense Master Cheatsheet (Lecture 6 Focus)

**Q1: "WHAT IS THE PRIMARY ADVANTAGE OF ACTOR-CRITIC OVER PURE POLICY GRADIENTS (REINFORCE)?"**

**Answer:** "REINFORCE relies on empirical Monte Carlo returns, requiring complete trajectory rollouts. This introduces immense variance, as a single disturbance late in the episode pollutes credit assignment for all earlier steps. Actor-Critic introduces a learned Critic  $V_\phi(s)$  that evaluates states via 1-step Temporal Difference bootstrapping ( $r + \gamma V(s')$ ), drastically reducing variance and enabling stable, sample-efficient learning in contact-rich tasks."

**Q2: "EXPLAIN THE BIAS-VARIANCE TRADEOFF BETWEEN MONTE CARLO EVALUATION AND 1-STEP TD LEARNING."**

**Answer:** "Monte Carlo evaluation has zero bias because it measures true realized rewards, but suffers from extreme variance due to compounding stochastic transitions. 1-step TD learning has minimal variance because it only looks one step ahead, but introduces bias because it relies on the Critic's own function approximation  $\hat{V}(s_{t+1})$ , which may be inaccurate early in training."

**Q3: "WHY IS GAE- $\lambda$  UNIVERSALLY PREFERRED FOR CONTINUOUS ROBOTIC MANIPULATION LIKE TOMATO SLICING?"**

**Answer:** "Robotic cutting involves abrupt physical regime shifts (e.g., non-contact approach  $\rightarrow$  elastic skin indentation  $\rightarrow$  rupture  $\rightarrow$  flesh slicing). Pure 1-step TD is too myopic to anticipate delayed fruit deformation, while pure Monte Carlo is overwhelmed by contact noise. GAE- $\lambda$  exponentially weights multi-step returns with parameter  $\lambda = 0.95$ , providing the optimal balance: it reacts quickly to localized rupture events while maintaining a sufficient temporal horizon to optimize smooth compliant trajectories."